2 - Recursión

Tabla de contenidos

1 Introducción 1
2 Cuenta regresiva 1
2.1 Versión comentada 3
2.2 ¡Qué no falte el caso base! 4
3 Factorial 5
3.1 Versión comentada 6
4 Letra chica 7

1 Introducción

En el apunte anterior vimos que en Python las funciones son ciudadanos de primera clase. Esto significa que una función es un objeto, al igual que un número, una cadena de texto o incluso un diccionario anidado con una estructura bastante enredada. Gracias a esto, no solo podemos inspeccionar sus atributos o hacer que una función llame a otras, sino que también es posible que una función cree y retorne nuevas funciones.

Además de los casos ya mencionados, también es posible que una función se llame a sí misma. Es decir, que en el cuerpo de la definición de esa función se incluya una llamada a la función que se está definiendo (🤯). A este tipo de funciones se las llama **recursivas**, y la técnica en sí recibe el nombre de **recursión**.

Aunque pueda sonar extraño que una función se invoque a sí misma, en programación existen problemas para los que funciones recursivas resultan una solución natural y efectiva.

2 Cuenta regresiva

Supongamos que queremos crear una función que imprima una cuenta regresiva desde un número entero dado hasta el 0. Es decir, la función se debería comportar de la siguiente manera:

```
regresiva(3)
```

```
3
2
1
0
```

```
regresiva(5)
```

```
5
4
3
2
1
0
```

Una implementación **no recursiva** es la siguiente, basada en un bucle `while`:

```
def regresiva(n):
    while n > 0:
        print(n)
        n = n - 1
```

Otra alternativa, que usa un bucle `for` y un `range` es:

```
def regresiva(n):
    for i in range(n, -1, -1):
        print(i)
```

Ambas implementaciones resultan en funciones que se comportan correctamente y producen el resultado deseado.

Sin embargo, existe una **alternativa recursiva** para obtener la secuencia de números deseados.

```
def regresiva(i):
    print(i)
    if i > 0:
        regresiva(i - 1)
    else:
        return
```

Líneas 3-4

El **caso recursivo**: Es la condición donde la función se llama a sí misma. En este caso, `regresiva` vuelve a llamarse a sí misma solo cuando el número impreso sea mayor que 0. La llamada se hace pasando como argumento el valor `i - 1`, lo que nos acerca progresivamente al **caso base**.

Líneas 5-6

El **caso base**: Es la condición que **detiene la recursión**. Cuando se cumple, la función devuelve un valor sin volver a llamarse a sí misma, lo que evita que la ejecución continúe de manera infinita.

Ambos casos trabajan en conjunto. Por un lado, el caso recursivo es la parte de la función que se llama a sí misma, pero con una entrada modificada que se acerca progresivamente al caso base. Por

el otro, el caso base es la condición que detiene las llamadas recursivas, representando la versión más simple del problema que puede ser resuelta directamente.

```
regresiva(3)
```

```
3
2
1
0
```

```
regresiva(5)
```

```
5
4
3
2
1
0
```

> **i** Versión con `return` implícito
>
> La función recursiva `regresiva` usa `return` para terminar su ejecución devolviendo un `None` implícito en el caso base. Otra forma de implementar la misma función es omitiendo el `return`:
>
> ```
> def regresiva(i):
>     print(i)
>     if i > 0:
>         regresiva(i - 1)
> ```
>
> **Línea 4**
>
> Esta línea marca el **caso recursivo** y es idéntica a la anterior.
>
> **Línea 5**
>
> Acá no se usa `return` para devolver `None`, directamente no se escribe nada. El efecto es el mismo y constituye el **caso base**. En Python, si una función no incluye ningún `return`, se comporta como si al final hubiera un `return` o `return None`, es decir, la función termina y devuelve `None`.

2.1 Versión comentada

Para entender mejor el funcionamiento de la función recursiva `progresiva`, se pueden incorporar unos `print` en el cuerpo de la misma. En el ejemplo de abajo se muestra un `print` justo cuando la función es llamada, y otro cuando la función vuelve a invocarse a sí misma.

```python
def regresiva(i):
    print(f"regresiva({i})")
    print(i)
    if i > 0:
        print(f"regresiva({i}) --> regresiva({i -1})")
        regresiva(i - 1)

regresiva(2)
```

```
regresiva(2)
2
regresiva(2) --> regresiva(1)
regresiva(1)
1
regresiva(1) --> regresiva(0)
regresiva(0)
0
```

Línea 1

Se ejecuta `regresiva` con `i = 2`

Línea 2

`regresiva(2)` imprime `2`

Línea 3

`regresiva(2)` llama a `regresiva(1)`

Línea 4

Se ejecuta `regresiva` con `i = 1`

Línea 5

`regresiva(1)` imprime `1`

Línea 6

`regresiva(1)` llama a `regresiva(0)`

Línea 7

Se ejecuta `regresiva` con `i = 0`

Línea 8

`regresiva(0)` imprime `0`

2.2 ¡Qué no falte el caso base!

Cuando escribimos una función recursiva es **fundamental implementar el caso base**, que determina la condición donde la función deja de llamarse a sí misma y comienza a devolver un valor.

Debajo se muestra una implementación incorrecta de la función `regresiva` como función recursiva. La misma tiene el caso recursivo, donde la función se llama a sí misma, pero le falta el caso base, que es el que detiene esa cadena de llamadas. Como resultado, la función sigue contando números, incluso negativos, y Python termina frenando la ejecución con un `RecursionError`. Este error aperece cuando la profundidad de la pila de llamadas recursivas supera un límite predeterminado y, como regla general, indica que hay un problema en nuestra recursión.

```
def regresiva(i):
    print(i)
    regresiva(i - 1)

regresiva(1)
```

```
1
0
-1
-2
...
-988
-989
Traceback (most recent call last):
  File "<python-input-5>", line 1, in <module>
    regresiva(1)
    ~~~~~~~~~^^^
  File "<python-input-2>", line 3, in regresiva
    regresiva(i - 1)
    ~~~~~~~~~^^^^^^^
  File "<python-input-2>", line 3, in regresiva
    regresiva(i - 1)
    ~~~~~~~~~^^^^^^^
  File "<python-input-2>", line 3, in regresiva
    regresiva(i - 1)
    ~~~~~~~~~^^^^^^^
  [Previous line repeated 988 more times]
RecursionError: maximum recursion depth exceeded
```

3 Factorial

Un ejemplo clásico para estudiar cómo funciona la recursión en programación es el cálculo del factorial. El factorial de un número positivo n se define como:

$$n! = n \times (n-1) \times (n-2) \times \cdots \times 2 \times 1$$

Es decir, el factorial de un número es el producto de todos los enteros desde 1 hasta n. Por convención, además, se establece que $0! = 1$.

Una forma de calcular el factorial de un número sin recurrir a la recursión es la siguiente:

```
def factorial(n):
    resultado = 1
    for i in range(2, n + 1):
        resultado = resultado * i
    return resultado

factorial(6)
```

```
720
```

Sin embargo, y dado que el factorial admite la siguiente expresión como función recursiva,

$$n! = \begin{cases} 1 & \text{para} \quad n = 0 \quad \text{o} \quad n = 1 \\ n \times (n-1)! & \text{para} \quad n \geq 2 \end{cases}$$

se puede implementar en Python de la siguiente manera:

```
def factorial(n):
    if n <= 1:
        return 1
    else:
        return n * factorial(n - 1)
```

Línea 3

Caso base.

Línea 5

Caso recursivo. La función hace uso del resultado que ella misma produce.

Podemos ver algunos ejemplos:

```
print("factorial(1):", factorial(1))
print("factorial(0):", factorial(0))
print("factorial(6):", factorial(6))
print("factorial(10):", factorial(10))
```

```
factorial(1): 1
factorial(0): 1
factorial(6): 720
factorial(10): 3628800
```

3.1 Versión comentada

La función `regresiva` nos sirvió como primera aproximación a la recursión. Sin embargo, en ella, no se hace uso del resultado que la misma función produce. En cambio, con `factorial` sí se utiliza

el valor producido en el caso recursivo, lo que la vuelve más interesante para analizar cómo se van encadenando los distintos pasos.

```python
def factorial(n):
    print(f"factorial({n})")
    if n <= 1:
        salida = 1
    else:
        print(f"factorial({n}) -> factorial({n - 1})")
        salida = n * factorial(n - 1)

    print(f"> factorial({n}) devuelve {salida}")
    return salida

factorial(4)
```

```
factorial(4)
factorial(4) -> factorial(3)
factorial(3)
factorial(3) -> factorial(2)
factorial(2)
factorial(2) -> factorial(1)
factorial(1)
> factorial(1) devuelve 1
> factorial(2) devuelve 2
> factorial(3) devuelve 6
> factorial(4) devuelve 24
24
```

El proceso arranca con la llamada a `factorial(4)`. Para calcular su resultado, la función necesita antes el valor de `factorial(3)`. Esa, a su vez, depende de `factorial(2)`, que depende de `factorial(1)`. Finalmente, cuando llegamos a `factorial(1)`, entramos en el **caso base**, que devuelve un resultado sin hacer más llamadas.

Por eso primero vemos cómo se acumula la **pila de llamadas**, que crece paso a paso hasta llegar al caso base. Recién ahí comienza el camino de regreso: cada llamada se va resolviendo, utilizando el valor devuelto por la llamada anterior, hasta llegar de nuevo a `factorial(4)`, donde se completa el cálculo y obtenemos el resultado final.

4 Letra chica

Cada vez que una función se invoca a sí misma, crea un nuevo **contexto de ejecución** (con sus propias variables locales y el punto al que debe regresar) y lo apila sobre el anterior en una estructura que se llama **pila de llamadas** (del inglés, *call stack*). Por lo tanto, cuando llamamos a `factorial(4)`, apilamos cuatro contextos de ejecución distintos, uno por cada llamada que va quedando pendiente, antes de empezar a devolver resultados.

Este apilamiento **no es gratis**, consume memoria de nuestra computadora. Aunque para `factorial(4)` el consumo de memoria es insignificante, si intentáramos calcular un factorial muy grande, la pila podría crecer hasta agotar la cantidad de memoria disponible en nuestra computadora. Para evitar que las funciones recursivas causen una falla estrepitosa, Python establece un límite en la profundidad de la recursión. Al alcanzarlo, el intérprete se detiene con la excepción `RecursionError` que nos proteje de una ejecución recursiva infinita o excesivamente profunda.

Dado su potencial alto consumo de memoria y la sobrecarga que genera gestionar la pila de llamadas, una implementación recursiva no suele ser la más eficiente. De hecho, su equivalente iterativo suele ser más eficiente y además evita los riesgos asociados al desbordamiento de la pila. Sin embargo, el principal valor de la recursión reside en la **claridad conceptual** para modelar problemas de naturaleza inherentemente recursiva.